

EVA — Emergent Vector Architecture

Что это?

EVA — это экспериментальная нейросетевая архитектура для понимания и генерации русского языка. Она принципиально отличается от всех существующих LLM тем, что **не угадывает следующее слово по вероятности, а навигационно перемещается в многомерном пространстве смыслов.**

Обычная LLM делает одно: смотрит на предыдущие токены и вычисляет, какой токен вероятнее всего будет следующим. Это работает, но это трюк — модель не понимает, что слова означают, она просто знает, с какими другими словами они часто встречаются. EVA пытается построить **внутреннее пространство**, где положение каждого слова определяется его смыслом, а переходы между словами — это траектории движения в этом пространстве.

1. Базовый принцип: координатная навигация вместо next-token prediction

Проблема LLM

LLM (GPT, LLaMA, Mistral, и т.д.) решают задачу:

$$P(\text{token}_t \mid \text{token}_1, \text{token}_2, \dots, \text{token}_{\{t-1\}})$$

То есть: «глядя на все предыдущие токены, вычисли распределение вероятностей для следующего». Это переформулировка задачи сжатия (compression) — модель учится минимизировать количество бит, необходимых для кодирования текста.

Результат: LLM отлично генерируют правдоподобный текст, но не имеют представления о том, что́ они говорят. Они не знают, что «король» и «королева» отличаются по полу, а «король» и «монарх» — почти синонимы. Они не могут отличить истинное утверждение от ложного, если оба одинаково вероятны в тренировочных данных.

Подход EVA

EVA решает другую задачу:

1. Каждый токен (символ, часть слова или целое слово) отображается в точку 384-мерного пространства — **символическую координату**
2. Чтение текста — это траектория: последовательность точек, через которые модель проходит
3. Понимание — это способность предсказать, **в каком направлении** будет следующая точка
4. Генерация — это движение вдоль градиента **поля аттракторов**, которое накапливает знание о типичных траекториях

Это принципиально иная парадигма. Модель не спрашивает «какое слово вероятнее всего?»; она спрашивает **«куда я должен двигаться в пространстве смыслов, чтобы оказаться там, где имеет смысл быть?»**

Почему 384 измерения?

Пространство должно быть достаточно большим, чтобы вместить все смысловые различия: род, число, падеж, время, наклонение, модальность, оценку, интенсивность, и т.д. Если бы у нас было 2D-пространство, слова бы неизбежно накладывались друг на друга. 384 измерения — это компромисс между выразительностью и вычислительной эффективностью.

Эмпирическая проверка: для 6 масштабов (char → morph → word → phrase → sentence → discourse) нужно минимум ~64 оси на масштаб. $6 \times 64 = 384$.

2. Словарь: BPE + границы

BPE-токенизация

Сырой текст состоит из символов, но отдельные символы не несут достаточно смысла. Вместо них используются BPE-токены (Byte-Pair Encoding): алгоритм ищет самые частые пары символов и объединяет их в один токен. Повторяя этот процесс, мы получаем словарь из 4096 токенов, где каждый токен представляет часто встречающуюся последовательность символов (от одной буквы до целого слова).

Пример: «искусственный» может быть закодирован как [«искусств», «енный»] или [«искусственный»] в зависимости от того, как часто встречается слово.

Boundary-токены

Поверх BPE модель использует 5 специальных токенов границ:

- **GAP** (4096) — заполнитель для выравнивания
- **WO / WC** (4097-4098) — Word Open / Word Close — отмечают границы слов
- **SO / SC** (4099-4100) — Sentence Open / Sentence Close — отмечают границы предложений

Но! В основном корпусе (Phase 2) границы не вставляются в последовательность. Вместо этого создаётся параллельный массив меток: для каждой позиции — 0 (начало слова), 1 (середина), 2 (конец). Это позволяет модели учиться распознавать границы слов **самостоятельно**, без явных маркеров.

Проблема Phase 1

Первая фаза обучения использовала char-уровень (155 символов, включая буквы, цифры, пунктуацию). Модель обучалась на 94.7М токенах и вышла на плато при $CE \approx 1.3$, $accuracy \approx 58\%$. Проблема: 155 токенов — слишком мало; модель выучила

все возможные пары очень быстро и перестала учиться. BPE с 3890 реальными токенами даёт принципиально больше сигнала.

3. MultiScaleRoPE — многочастотное позиционирование

Стандартный RoPE

В трансформерах нужно как-то сообщить модели, на какой позиции находится каждый токен (порядок слов критичен для смысла). Rotary Position Embedding (RoPE) кодирует позицию через вращение пар координат: для каждой пары (d_{2k}, d_{2k+1}) вычисляется угол θ_{pos} , и вектор поворачивается на этот угол.

Проблема: в стандартном RoPE все пары координат вращаются с частотами, равномерно распределёнными от θ_{min} до θ_{max} . Это значит, что все размерности видят примерно один и тот же диапазон позиций.

Многомасштабность EVA

EVA использует логарифмическую шкалу: первые пары координат имеют минимальную частоту ($\theta=500 \rightarrow$ период ≈ 3140 позиций), они «видят» только локальный контекст — различают соседние символы. Последние пары имеют максимальную частоту ($\theta=200000 \rightarrow$ период ≈ 1.26 млн позиций), они «видят» весь корпус целиком — различают структуру уровня документа.

Это даёт модели возможность одновременно:

- Различать порядок букв внутри слова (высокие частоты)
- Различать порядок слов в предложении (средние частоты)
- Различать порядок предложений в дискурсе (низкие частоты)

Все 192 пары координат работают одновременно. Модель сама учится, какие частоты использовать для какой задачи.

4. GroupedScaleAttention — внимание на разных масштабах

Проблема

Стандартный 多头注意力 (multi-head attention) использует все головы одинаково. Каждая голова может специализироваться на чём-то своём, но нет механизма, который бы гарантировал, что какая-то голова будет смотреть на char-уровень, а какая-то — на discourse.

Решение

EVA делит 24 головы на 6 групп по 4 головы. Каждая группа имеет свою **отдельную проекционную матрицу** W_O ($64 \rightarrow 384$) — то есть каждая группа учится по-своему проецировать внимание обратно в скрытое пространство.

Изначально каждая группа получает «предпочтение» определённого масштаба, которое меняется от слоя к слою:

Слой

Группы

- | | |
|-------|--|
| 0-1 | В основном char (70%) + немного morph (20%) + word (10%) |
| 2-3 | Смесь char/morph/word |
| 4-5 | В основном word/phrase |
| 6-7 | Смесь phrase/sentence |
| 8-9 | В основном sentence/discourse |
| 10-11 | Почти полностью discourse (55-65%) |

Эти предпочтения — не жёсткая фиксация, а мягкое начальное смещение (soft gate). Модель может их изменить в процессе обучения, если найдёт лучшую стратегию.

Почему это важно

В длинном тексте на разных уровнях происходят разные процессы:

- На char-уровне: «приставка -пре- означает высокую степень»
- На word-уровне: «существительное в родительном падеже после отрицания»
- На discourse-уровне: «это предложение — контраргумент к предыдущему»

Группировка голов гарантирует, что у модели есть вычислительные ресурсы для каждого уровня одновременно, а не надежда, что они как-нибудь сами распределятся.

5. Три residual stream — разделение информации

Проблема одного потока

В обычном трансформере:

$$x_l = x_{l-1} + \text{Attention}(\text{LayerNorm}(x_{l-1})) + \text{FFN}(\text{LayerNorm}(...))$$

Всё, что модель узнала на предыдущих слоях, складывается в один вектор x . Проблема: на слое 10, когда модель обсуждает глобальную тему, информация о том, какой символ был на позиции 3, полностью перезаписана. Модель **вынуждена** либо помнить детали (и жертвовать ёмкостью для глобального), либо забывать детали (и терять точность).

Решение: три потока

EVA ведёт три отдельных residual stream (остаточных потока):

1. **residual1** — для локальной структуры (char → morph)

- Обновляется сильно на ранних слоях ($\alpha=0.3$), слабо на поздних ($\alpha=0.05$)
- Хранит: какая буква была на позиции 3, какой корень у этого слова

2. **residual2** — для синтаксиса (word → phrase)

- Обновляется средне везде ($\alpha=0.1 \rightarrow 0.3 \rightarrow 0.15$)
- Хранит: это подлежащее или сказуемое, какие слова образуют группу

3. **residual3** — для глобальной структуры (sentence → discourse)

- Обновляется слабо на ранних слоях ($\alpha=0.05$), сильно на поздних ($\alpha=0.30$)
- Хранит: о чём это предложение, как оно связано с предыдущим

Динамика α по слоям:

Layer:	0	1	2	3		4	5	6	7		8	9	10	11
residual1:	30	30	30	30		10	10	10	10		5	5	5	5
residual2:	10	10	30	30		30	30	30	30		15	15	15	15
residual3:	5	5	5	5		15	15	15	15		30	30	30	30
	— char/morph —					— word/phrase —					— discourse —			

Каждый слой **читает из всех трёх потоков** (суммирует их перед attention/FFN), а пишет в каждый с разным коэффициентом. Это значит, что слой 10, обрабатывая discourse, всё ещё имеет доступ к char-уровневой информации через residual1 — просто с меньшим весом.

6. AttractorField — поле аттракторов

Что это и зачем

AttractorField — это отдельная структура памяти, которая **накапливает знания о траекториях** в процессе обучения. Это альтернатива хранению знаний в весах модели.

В LLM знания «зашиты» в матрицы весов. Чтобы добавить новое знание, нужно переобучать модель (fine-tuning, LoRA и т.д.). В EVA знания хранятся в аттракторах — их можно добавлять без изменения весов.

Как устроен один аттрактор

Каждый аттрактор — это тройка:

- **μ_a** (center) — точка в 384-мерном пространстве, представляющая центр кластера похожих состояний
- **w_a** (count) — счётчик: сколько раз модель прошла через этот аттрактор
- **r_a** (refractory) — рефрактерный вектор: типичное направление выхода из аттрактора (куда обычно движется следующая точка)

Hebbian update — как аттракторы учатся

Когда модель читает текст, она проходит через последовательность скрытых состояний $h_1, h_2, \dots, h_L$. Для каждой пары (h_t, h_{t+1}) :

1. Найти ближайший аттрактор к h_t (по евклидову расстоянию)
2. Увеличить его счётчик на 1
3. Сдвинуть центр в сторону h_t : $\mu \leftarrow \mu + (lr / count) \cdot (h_t - \mu)$
— чем больше count, тем меньше сдвиг (асимптотическая сходимость)
4. Обновить рефрактер: $r \leftarrow r + lr_{refract} \cdot ((h_{t+1} - h_t) - r)$
— EMA направления движения

Шаг 3 критичен: **count-normalized update** означает, что первые несколько проходов через аттрактор сильно двигают его центр, а каждая последующая —

всё слабее. Это гарантирует, что центр сходится к истинному среднему всех точек в кластере, а не бесконечно дрейфует.

Потенциал поля

Поле $P(z)$ в точке z определяется как:

$$P(z) = \sum_a w_a \cdot \exp(-\|z - \mu_a\|^2 / 2\sigma^2)$$

То есть: суммируем вклады всех аттракторов, взвешенные по их счётчику и убывающие с расстоянием. Это гладкая функция, определённая во всём пространстве $\mathbb{R}^{384}$.

Градиент потенциала $\nabla P(z)$ указывает направление, в котором плотность аттракторов растёт наиболее быстро. Это направление — **куда имеет смысл двигаться** в пространстве смыслов.

Генерация через поле

При генерации у модели есть три режима:

1. **Standard**: TrajectoryBoundaryPredictor предсказывает nxt вектор непосредственно из скрытого состояния h_t
2. **Attractor-guided**: AttractorField. $\text{nxt_direction}(z)$ вычисляет направление как комбинацию:
 - $\eta \cdot (\mu^* - z) / \|\mu^* - z\|$ — притяжение к ближайшему аттрактору
 - $(1-\eta) \cdot r^* / \|r^*\|$ — продолжение в типичном направлении выхода (где $\eta=0.7$ — баланс между консервативностью и инновацией)
3. **Gradient ascent (Phase 4)**: $z_{t+1} = z_t + \eta \cdot \nabla P(z_t) + \text{шум}$
— Ланжевеновская динамика на потенциале поля

Финальные логиты вычисляются через расстояние от предсказанной координаты z_{pred} до всех символов в embedding table, взвешенные через MetaWeighter:

```
logits_know = decoder(z_pred)
logits_conc = -||z_pred - E|| · (1.0 + concept_score)
logits_contr = -||z_pred - E|| · (1.0 - contra_score · 0.5)
final = softmax(W_know · logits_know + W_conc · logits_conc + W_contr · logits_contr)
```

Top-20 sampling + repetition penalty.

7. Boundary detection — распознавание границ слов

Задача

В русском языке границы слов неочевидны на уровне BPE. Один BPE-токен может быть частью слова («пре-», «-крас-», «-ный»), а одно слово — состоять из множества токенов. Модель должна научиться определять: «этот токен — начало нового слова», «этот — продолжение», «этот — конец».

BoundaryDetectionHead

Небольшой MLP ($384 \rightarrow 64 \rightarrow 3$), который из скрытого состояния h_t вычисляет три логита: [word_start, word_inside, word_end]. Обучается supervised на размеченном корпусе (60.2M токенов с метками 0/1/2).

Корпус с метками

Создаётся автоматически: исходный текст разбивается на предложения (по точке

- заглавная), предложения — на слова (по пробелам). Каждое слово BPE-кодируется, и каждому токenu присваивается метка:
- Если BPE-токен — первый в слове $\rightarrow$ label 0 (start)
- Если BPE-токен — в середине слова $\rightarrow$ label 1 (inside)

- Если BPE-токен — последний в слове → label 2 (end)
- Если слово состоит из 1 BPE-токена → label 2 (start + end одновременно)

Пунктуация обрабатывается отдельно: каждый знак — отдельное «слово» длины 1, получает label 2.

WordWeightEncoder

Параллельно с supervised boundary detection работает unsupervised механизм: WordWeightEncoder вычисляет важность каждого слова на основе attention к нему. L_align loss (MSE = 0.05) принуждает эти два сигнала согласовываться: char_inside (из boundary detection) должен совпадать с word_importance (из WordWeightEncoder). Это bridge между supervised и unsupervised обучением.

8. Архитектура потерь (composite loss)

Модель обучается на смеси из 5+ функций потерь:

1. Cross-entropy (W=1.0)

Главная потеря. Угадать следующий BPE-токен среди 4101 возможных. Special-токены (PAD, UNK, BOS, EOS, GAP, SO, SC) маскируются — модель не учится их предсказывать.

2. Trajectory loss (W=0.05)

MSE между предсказанным h_{t+1} -вектором и реальным сдвигом $h_t \rightarrow h_{t+1}$. Учит модель предсказывать направление в пространстве, а не просто токен.

3. Boundary loss (W=0.1)

Cross-entropy на метки границ слов (0/1/2). Учит BoundaryDetectionHead распознавать границы слов в потоке BPE-токенов.

4. L_align loss (W=0.05)

MSE между char_inside (supervised boundary) и word_importance (unsupervised WordWeightEncoder). Согласует два уровня анализа.

5. Attractor consistency (W=0.01)

MSE между скрытым состоянием h_t и его ближайшим аттрактором. Принуждает модель производить состояния, которые естественно группируются в аттракторы. Включается после 1000 шагов warmup.

6. Attractor diversity (W=0.01)

Штраф за коллинеарность аттракторов: чем более ортогональны центры аттракторов, тем меньше loss. Предотвращает схлопывание всех аттракторов в одну точку.

9. Данные

Сырой корпус

- 173 MB русского текста (смесь новостей, литературы, разговоров)
- ~1.2 млн строк

Char-level корпус (Phase 1)

- 94.7M токенов, 155 уникальных символов

- Сохранён: `full_corpus_ids.npy`

ВРЕ корпус (Phase 2)

- 29.4М токенов, 3890/4096 реальных ВРЕ-токенов (оставшиеся — и зарезервированные)
- Сохранён: `full_corpus_bpe.npy`

Boundary корпус (Phase 2)

- 60.2М токенов (ВРЕ + 12М пар WO/WC)
 - Параллельный массив меток: 12М слов с метками 0/1/2
 - Сохранён: `full_corpus_bpe_boundary.npy` + `_labels.npy`
-

10. Параметры и производительность

Характеристика	Значение
Всего параметров	20,473,064
Размерность (d_model)	384
Слоёв	12
Голов внимания	24 (6 групп × 4)
Размер головы	16
Размер FFN	512 (SwiGLU)
Vocab (BPE)	4096 + 5 boundary = 4101
Max seq_len	2048
Batch size	8
Seq length	64
Скорость обучения	3 steps/s на MX550
VRAM	0.7-1.4 GB из 2.1 GB

Характеристика	Значение
Время Phase 2	~18 часов на 200K шагов

Распределение параметров

Компонент	Параметров	%
Embedding (4101 × 384)	1,574,784	7.7%
Attention (12 × 589,824)	7,077,888	34.6%
FFN (12 × 589,824)	7,077,888	34.6%
TrajectoryPredictor	394,240	1.9%
ResidualHead	484,608	2.4%
WordValence	1,099,520	5.4%
Остальные heads	~2M	~10%

11. Сравнение с GPT-2

Аспект	GPT-2 (124M)	EVA (20.5M)
Параметры	124M	20.5M (×6 меньше)
Размерность	768	384
Слои	12	12
Головы	12	24
Attention	Standard	GroupedScale (6×4)
RoPE	Нет	MultiScale (500→200K)
Residual	1 поток	3 потока
Позиции	Learned	MultiScaleRoPE
FFN	GELU ($x \rightarrow 3072 \rightarrow x$)	SwiGLU (384 → 512 → 384)
Norm	LayerNorm	RMSNorm
Память	В весах	Beca + AttractorField

Аспект	GPT-2 (124M)	EVA (20.5M)
Генерация	Argmax/sampling	Gradient in $P(z)$
Понимание	Статистическое	Координатно-навигационное

12. Фазы обучения

Phase 0 (завершена)

- Boundary detection на char-level корпусе
- 5000 шагов
- Проверка: работает ли boundary detection вообще

Phase 1 (завершена)

- Char-level pre-training (155 токенов)
- 60,000 шагов
- Результат: $CE \approx 1.3$, accuracy $\approx 58\%$
- Вывод: char-level слишком мал для meaningful обучения
- Модель выучила тривиальные пары букв (ча-ща, жи-ши) и упёрлась в плато

Phase 2 (текущая, ~18 часов)

- BPE-level training (3890 токенов)
- Границы слов через labels (0/1/2), не через маркеры в последовательности
- AttractorField: Hebbian update каждые 10 шагов
- Attractor loss: consistency + diversity (после 1000 шагов)
- Loss: $CE(1.0) + nxt(0.05) + boundary(0.1) + align(0.05) + attractor(0.01)$
- 200,000 шагов, $B=8$, $L=64$
- Checkpoint каждые 20,000 шагов

Phase 3 (план)

- Усилить AttractorField: сделать его полноценной частью обучения
- Добавить prototype vectors как learnable параметры
- Iterative refinement: КСА-цикл (коррекция латентного кода)

Phase 4 (план)

- Генерация через градиентный подъём по $P(z)$
 - Langevin dynamics: $z_{t+1} = z_t + \eta \cdot \nabla P(z_t) + \text{noise}$
 - Без учителя — генерация из поля, а не из токенов
-

13. Ключевые отличия от mainstream ML

1. **Не GPT, не BERT, не T5.** EVA не использует архитектуру decoder-only с causal attention как единственный механизм. Это custom архитектура с нуля.
2. **Не next-token prediction.** Основной сигнал обучения — CE loss — да, используется. Но он не единственный, и способ генерации в конечном итоге будет не через argmax , а через градиент поля.
3. **Память отдельно от весов.** AttractorField — это не слой модели в обычном смысле. Это отдельная структура с Hebbian update, которая растёт со временем и может быть пополнена независимо.
4. **Масштаб не главное.** 20.5M параметров против 1.5T+ у современных LLM — на 5 порядков меньше. Цель не «догнать GPT-4», а проверить гипотезу: можно ли с 20M параметров + поле аттракторов получить нетривиальное понимание языка.
5. **Генерация как физический процесс.** Переход от точки к точке в пространстве под действием потенциала — это метафора физического процесса

(частица в потенциальном поле). Не статистическая выборка, а детерминированный поток с шумом.

14. Технические ограничения

1. **Только русский язык.** Токенизатор обучен на русском корпусе. Для другого языка нужен новый BPE.
 2. **Ограниченный корпус.** 173 MB — это ничтожно мало по сравнению с LLM-датасетами (единицы TB). Но для проверки гипотезы достаточно.
 3. **Простая генерация.** Текущий генератор (top-20 sampling с repetition penalty) — временное решение. Полноценная генерация через $\nabla P(z)$ будет в Phase 4.
 4. **MX550 — слабая карта.** 2.1 GB VRAM, ~2 TFLOPS. На карте уровня RTX 4090 (24 GB, 82 TFLOPS) можно было бы поставить $d_model=768$, $L=64$, и получить ~50M параметров с пропорционально лучшим качеством.
 5. **Символьное поле не используется.** TensorPotentialField $([V,V,V])$ и WordValenceField реализованы, но не активированы в текущем обучении. Потенциально они дадут дополнительный сигнал, но требуют доработки.
-

15. HierarchicalAdditiveField — иерархическое аддитивное хранение

15.1. Проблема

AttractorField хранит точки (концепты), но не знает, **из чего они состоят**.

Слово "пришёл" — это не только точка в пространстве, но и сумма смыслов

"при-" + "-шёл-" + "-л". Без разложения модель видит только целое и не может понять, что "пришёл" и "прибежал" имеют общую приставку.

15.2. Решение

NAF раскладывает любой вектор z в сумму K суб-векторов:

$$z \approx v_0 + v_1 + \dots + v_K$$

Каждый v_k — самостоятельный концепт, который тоже может быть разложен.

K — переменное: сколько компонент нужно, чтобы объяснить z .

15.3. Как работает (простыми словами)

1. Берём вектор z (концепт)
2. Пытаемся извлечь из него первую составляющую: $v_0 \approx z$
3. Вычитаем: остаток $r_1 = z - v_0$
4. Из остатка извлекаем $v_1 \approx r_1$
5. Повторяем, пока остаток не станет пренебрежимо мал
6. STOP — сигнал остановки, когда остаток меньше порога

Гарантия точности: skip-connection ($v_k = \text{MLP}(r) + r$) даёт $v_k \approx r$ на первом же шаге обучения. Модель не борется за реконструкцию — она сразу учится **дробить** вектор на осмысленные части.

15.4. Множественные пути

Один и тот же z можно разложить по-разному (как $5 = 1+4 = 2+3$). Разный dropout в MLP даёт разные decomposition paths. Loss следит, чтобы все пути:

- реконструировали z (точность)
- были согласованы между собой (consistency)

- использовали разные разбиения (diversity)
- использовали как можно меньше компонент (sparsity)

15.5. Зачем это модели

Без HAF модель гадает: "следующий токен — X?".

С HAF модель знает:

- "Текущий концепт Z состоит из A и B"
- "A — это под-концепт, который я уже видел"
- "B указывает в сторону концептов P, Q, R"
- "Следующее слово должно быть связано с P или Q"

Это превращает **угадывание** в **навигацию по иерархии знания**.

ConceptHead (оценка важности концепта) и ContradictionHead (оценка противоречия) получают измеримые сигналы: плотность аттракторов вокруг $z \rightarrow$ важность, разброс направлений суб-аттракторов $\rightarrow$ противоречие.

16. Философский итог

EVA — это не попытка сделать «ещё один ChatGPT на коленке». Это попытка ответить на вопрос: **может ли языковая модель действительно понимать, а не просто статистически аппроксимировать?**

Гипотеза: да, может. Если:

- Слова заменить координатами в пространстве смыслов
- Последовательность заменить траекторией
- Знания хранить не в весах, а в поле аттракторов
- Генерацию делать не через $\arg\max$, а через градиентный подъём по потенциалу

20.5M параметров — достаточно, чтобы проверить эту гипотезу. Если она верна, масштабирование до 100M-1B параметров даст модель, которая принципиально отличается от LLM — не как «хуже/лучше», а как **другая парадигма**.